Документация и сборник промптов проекта App Forge (Eraser.io)

> 📝 **Вступительная заметка: Сравнение промптов и нюансы флагманских AI-моделей**
>
> **Оригинальные промпты автора** на Eraser.io создавались как быстрые архитектурные наброски и визуальные заметки для видеоурока.
>
> **Нужна ли строгая структура для флагманских моделей (Claude 3.5 / Opus / Claude Code)?** Современные топовые модели с высокими возможностями рассуждения ("Reasoning") способны додумывать контекст даже по фрагментарным промптам. Однако в продакшн-разработке подробная адаптация промпта критически важна по 3 причинам:
>
> 1. **Контроль архитектуры:** Без явных ограничений модель может самовольно изменить стек (например, добавить Supabase вместо CloudHike или переписать роутер).
> 2. **Исключение галлюцинаций в API:** Задаются точные имена таблиц (`projects` , `project_assets`), переменные окружения (`CLOUDHIKE_CLIENT_ID`) и структуры файлов.
> 3. **Граничные случаи (Edge Cases):** Задание лимитов (например, "до 5 файлов до 10 МБ", "тайм-аут 15 секунд") гарантирует написание защищенного кода.
>
> *Примечание по PDF-экспорту:* В данном документе использованы явные HTML-якоря (`<a id="..."></a>`), благодаря чему оглавление останется кликабельным при сохранении в PDF.

📋 Содержание

1. Глава 1: Конфигурация AI-Агента и Планирование (AI Agent Configuration)
2. Глава 2: Генерация экранов приложения и CloudHike Jobs (Generate Mobile App Screens)
3. Глава 3: Подключение бэкенда CloudHike в 1 клик (Generated App Backend)
4. Глава 4: Визуальное Инлайн-Редактирование (Inline Editing Feature)
5. Глава 5: Генерация и Трансформация Изображений с ImageKit (AI Image Transformation)
6. Глава 6: Попеременный предпросмотр Expo Go и QR-код (Expo Preview & QR Flow)

---

1. Глава 1: Конфигурация AI-Агента и Планирование (AI Agent Configuration)

🔹 Оригинальный промпт автора на Eraser:

```
Now, when the user confirms the build plan and screens, the AI Agent should start building
the mobile app screen one by one.
While building the screens, make sure we add an inline editing feature line, allowing
users to touch/hover and select any element on the screen. After selecting an element,
users should be able to edit text/inline mode or open a popup for editing.
```

```
Also, make all these colors dynamic and variable-based. This means users should be able to switch themes without regenerating all the screens from scratch.
Generates at least 5 themes, make sure colors generate a theme color package, logic to switch screens instantly and switch between themes whenever they want. This should work by updating the global CSS/theme variables instead of re-creating the entire app code. Theme color switcher options inside the middle of canvas structure to top.
Make sure every generated app screen looks professional, modern, and polished, with attention to even the smallest design details.
The generated mobile app should be fully functional, meaning users should be able to click buttons and navigate between all generated screens. To request backend is missing, display dummy message. Also, implement a database fallback option if backend is not available.
Most importantly, we are going to use Cloudhike as the default backend provider for generated mobile apps. We will explain the actual Cloudhike backend integration later.
For now, send local storage data to act as temporary data. Later, when users connect their Cloudhike account, the app will be able to switch to the actual Cloudhike Storage and Database without requiring major changes.
Make sure generated app and architecture support this future migration while keeping the current app fully functional with local storage.
```

⚡ Адаптированный системный промпт (Enhanced Prompt):

```
Role: Senior AI Agent Architect & React Native Engineer.
Task: Implement the step-by-step code generation workflow for physical mobile app screens using OpenAI Agent SDK / Claude Code.

Requirements & Constraints:
1. Sequential Generation: Upon plan confirmation, generate React Native screens iteratively (`app/index.tsx`, `app/[screen].tsx`).
2. Theme Engine:
   - Do NOT hardcode hex colors in JSX. Use CSS/Theme variables.
   - Inject 5 predefined theme packages (Mint Leisure, Harbor Night, Emerald, Sunset, Cyberpunk).
   - Theme switching MUST update active theme state globally without re-triggering LLM screen generation.
3. Interactive Canvas Readiness:
   - Attach `data-component-id` to every generated JSX element to support hover states and click-to-select inline inspector in the UI canvas.
4. Navigation & Fallbacks:
   - Ensure Expo Router links work between generated screens.
   - Use `AsyncStorage` / Local Storage as initial state fallback before CloudHike backend is attached.
```

---

2. Глава 2: Генерация экранов приложения и CloudHike Jobs (Generate Mobile App Screens)

🔹 Оригинальный промпт автора на Eraser:

```
Let's use cloudhike.dev Jobs service to implement and run App Screen generation background execution jobs.
```

```
For reference, we can read background execution jobs process inside doc:
https://cloudhike.dev/docs/jobs
```

⚡ Адаптированный системный промпт (Enhanced Prompt):

```
Task: Offload long-running AI mobile app screen generation tasks to CloudHike Jobs API to
prevent Serverless Execution Timeouts (Vercel 15s/60s limits).

Execution Architecture:
1. Server Endpoint (`/api/jobs/generate`):
   - Accepts prompt, project ID, and requested screens payload.
   - Triggers an asynchronous Job via CloudHike Jobs SDK/REST API using `JOB_API_URL` and
`CLOUDHIKE_API_KEY`.
2. Webhook Listener (`/api/jobs/webhook`):
   - Listens for job completion events from CloudHike.
   - Validates webhook secret payload.
   - Upon completion, updates `projects` status to 'completed' and saves generated code
tree into PostgreSQL.
3. Local Testing Resolution:
   - Use `NEXT_PUBLIC_APP_URL` or `ngrok` tunnel URL dynamically for webhook target
resolution during local development.
```

---

3. Глава 3: Подключение бэкенда CloudHike в 1 клик (Generated App Backend)

🔹 Оригинальный промпт автора на Eraser:

```
Now, lets implement the Generate App Backend feature to manage Database, Storage,
Authentication, Jobs, Email Services and other backend services.
We are integrating Cloudhike.dev (https://cloudhike.dev) as the default backend provider
for each generated mobile app. Instead of configuring separate services from
AppHike/Supabase, users should see an option to authenticate and connect their own
Cloudhike account in one click.
Users should simply authenticate with CloudHike, and then, using CloudHike MCP, the AI
Agent should automatically configure and deploy the necessary backend for their generated
mobile application.
Make sure the AI Agent handles implementing, deploying, and connecting these CloudHike
services to the generated mobile app so that the app can store real data, authenticate
users, and handle media files.
As developers, we first need to configure Client ID and Client Secret for our AppForge
platform so users can connect their own CloudHike accounts.
CloudHike Integration Documentation: https://cloudhike.dev/docs/connect
In the Workspace header, add a Connect Backend button. When users click it, show CloudHike
as the backend provider and allow them to authenticate and connect their CloudHike
account.
Also, move the existing Share option under the Export menu, and show a combined menu that
includes Share, Export, and another add to export option.
Once CloudHike is successfully connected, display a CloudHike Backend Connected status in
the Workspace, so users can clearly see that their app is connected to the backend.
Make sure the entire integration is secure, professional, user-friendly, and consistent
```

```
with our existing AppForge UI/UX design.

[DB Schema Note from Author]:
Currently, the Generated App backend database save everything as JSON or record. Avoid
saving everything as JSON and save auto create multiple Tables base on requirements and
Properly Structure the database for mobile app and then properly save each record in
respective table.
```

⚡ Адаптированный системный промпт (Enhanced Prompt):

```
Task: Implement One-Click CloudHike Backend Provisioning via MCP for Generated Apps.

Implementation Workflow:
1. Platform Auth Credentials:
   - Configure OAuth 2.0 Client credentials: `CLOUDHIKE_CLIENT_ID` and
`CLOUDHIKE_CLIENT_SECRET`.
2. Workspace UI Update:
   - Header: Add a prominent "Connect Backend" action button.
   - Header Menu Restructure: Consolidate "Share", "Download ZIP", and "Push to GitHub"
under an "Export & Share" dropdown.
3. MCP Provisioning Pipeline:
   - When user authorizes CloudHike: AI Agent invokes CloudHike Model Context Protocol
(MCP) server.
   - Automatically provisions PostgreSQL database, Object Storage buckets, and Auth
endpoints for the target app.
   - MUST generate normalized SQL tables (e.g., `habits`, `expenses`, `users`) via Drizzle
ORM instead of dumping unstructured raw JSON blobs into a single column.
4. Client SDK Injection:
   - Automatically inject generated `lib/cloudhike.ts` configuration file into the
generated app codebase containing the project's connection keys.
```

---

4. Глава 4: Визуальное Инлайн-Редактирование (Inline Editing Feature)

🔹 Оригинальный промпт автора на Eraser:

```
Let's implement the Inline Editing Feature.
For any generated app screen that the user wants to edit, they should be able to click the
Edit button at the top of the canvas to enable Edit Mode.
Once Edit Mode is enabled, users should be able to click on any component or element
within the screen to select and edit it.
For quick editing, show a popover near the selected component with primary options,
allowing users to make changes quickly. Users should also have an option to switch to
Advanced Edit Mode for more detailed customization.
Make sure the popover has a proper header with clearly organized editing options, so users
can easily navigate between different properties.
Display in-line editing options with categorized icons and clean spacing, making it easy
for users to see and modify properties such as text, colors, typography, spacing, layout,
borders, backgrounds, and other component-specific settings.
Also, include an Edit with AI option as user can modify the selected component using AI
instructions.
When a user selects any component, make sure that the selected element is automatically
```

```
attached to the Chat Sidebar prompt box. This allows users to request changes for that specific component directly through the AI chat.
For example, if the user selects a button, they should be able to type "Change the button background color to blue", and the AI Agent should understand which component to edit and update only that element.
Most importantly, make the entire editing experience feel fast, smooth, and interactive. When users manually edit properties or use AI to edit components, the changes should be updated immediately on the screen without requiring regeneration or reloading.
Keep the editing interface clean, professional, intuitive, and consistent with our existing AppForge UI/UX design.
```

⚡ Адаптированный системный промпт (Enhanced Prompt):

```
Task: Build Visual Element Selector and Point-to-Point AI Component Editing on Workspace Canvas.

Inspector Components & Logic:
1. Mode Switcher: Toggle canvas between 'Interact Mode' and 'Edit Mode'.
2. Selection Interceptor:
   - Clicking an element with `data-component-id` highlights it with a blue bounding box.
   - Spawns a Floating Quick Popover near the target element.
3. Quick & Advanced Inspector:
   - Direct Manual Controls: Text value editor, Color Picker (Text, BG, Border), Spacing sliders (Padding/Margin), Flex alignment.
   - Advanced Panel: Opens deep style editor sidebar.
4. "Edit with AI" Contextual Link:
   - Automatically injects selected `componentId` and JSX snapshot into the Chat Sidebar input context.
   - Prompt Handler: Modifies strictly the targeted AST node in the component file without re-generating unaffected screens or layout wrappers.
```

---

5. Глава 5: Генерация и Трансформация Изображений с ImageKit (AI Image Transformation)

🔹 Оригинальный промпт автора на Eraser:

```
Now, let's implement the ImageKit AI Transformation feature.
For AI generated images used in the mobile app, we will use ImageKit to generate, manage, and transform images.
Also, make sure to integrate AI Image Transformation into our existing Mobile App Builder workflow so users can select any image component and apply AI-powered transformations directly.
When a user selects an image, in Edit Mode, show options to Replace Image, Generate AI Image, and Apply AI Transformation.
Users should be able to enter a prompt describing how they want to transform the selected image, and ImageKit should handle the transformation.
Make sure the transformed image is immediately updated in the App Preview and the generated mobile app without requiring users to regenerate the entire screen.
Also, ensure that the application handles image assets efficiently with URL-based image transformations and optimization.
```

```
Keep the ImageKit integration smooth, professional, and consistent with our existing
AppForge Inline Editing UI/UX.
Documentation: https://imagekit.io/docs/image-transformation
```

⚡ Адаптированный системный промпт (Enhanced Prompt):

```
Task: Integrate ImageKit API for Real-Time URL-Based AI Image Generation and
Transformations inside Mobile Apps.

Implementation Specifications:
1. Environment Configuration:
   - Store `IMAGEKIT_PRIVATE_KEY` and `IMAGEKIT_URL_ENDPOINT`.
2. Dynamic URL Generation Helper:
   - Prompt Image Generation: `https://ik.imagekit.io/[YOUR_ID]/ik-gen-img?prompt=
[URL_ENCODED_PROMPT]`
   - Transformations: Dynamic cropping (`tr=w-300,h-300,fo-auto`), background removal
(`tr=e-remove_bg`), format optimization.
3. Canvas Image Inspector:
   - When an image element is selected in Edit Mode, provide actions: "Replace Image",
"Generate AI Image", "Remove Background".
   - Direct live update of `src` attribute on canvas without full app re-render.
```

---

6. Глава 6: Попеременный предпросмотр Expo Go и QR-код (Expo Preview & QR Flow)

🔹 Оригинальный промпт автора на Eraser:

```
Now, let's make some changes to the App Preview section.
First, remove the QR Code section from the bottom and make the App Preview screen bigger
to provide more space for viewing and testing the generated app.
At the top of the App Preview section, add a Scan QR Code option so users can scan their
generated app on a real device. When the user clicks this option, show a popup modal
displaying the QR code, which they can scan using the Expo Go app.
Now, let's implement the Expo Preview functionality so users can preview their generated
mobile apps on actual devices by scanning the QR code through Expo Go.
Make sure the generated app structure and bundled files are properly processed, including
navigation, UI components, and theme configuration.
Most importantly, when users make changes to any screen (through inline editing, manual
editing, or AI-powered editing), those changes should be updated immediately in the
running app through live updates, without requiring users to rescan the QR code or reload
the app.
Ensure the Expo preview integration is smooth, fast, and ready to use, with proper loading
states, error handling, and user handling.
Keep the App Preview section clean, professional, spacious, and consistent with our
existing AppForge UI/UX design.

The QR code comes from Expo Snack, targeting SDK 54. AppForge sends the generated app code
to Snack, and Snack gives back a project URL (exp://snack...). We turn that URL into a QR
code. When you scan it with the Expo Go app on your phone, the app runs natively. The
Snack Session keeps the device live-synced to the browser. Generated apps use Expo Router
```

```
navigation instead of Expo Router because it runs more reliably in Snack. That was the
magic behind how Expo works.
```

⚡ Адаптированный системный промпт (Enhanced Prompt):

```
Task: Build Expo Snack Bundling Engine and Real-Time Physical Device Preview Pipeline via
QR Code.

Technical Specifications:
1. Bundler API Route (`/api/expo/bundle`):
   - Aggregates all generated React Native screens, components, assets, and theme state
into an Expo Snack-compatible JSON payload targeting Expo SDK 54.
2. QR Code Popup Modal:
   - Relocate fixed bottom QR code to a top bar "Scan QR Code" button.
   - Clicking button opens modal with dynamic QR rendering the `exp://snack.expo.dev/...`
project URL.
3. Live Sync Engine:
   - Maintain active Expo Snack WebSocket session.
   - When inline edits or AI prompt updates occur, push code differential to Snack session
to trigger instant hot-reload on physical iOS/Android devices running Expo Go without
requiring re-scan.
```

Документ полностью обновлен и оптимизирован для безупречной навигации в PDF-формате.